Lab 2 — Regression Selection & Prioritization

Course: Principles & Practices in Software Testing (DV033G)
Pair: Lena Edlind, Jakob Cederblad
Target Scenario: `feature/weekend-surcharge` (modifying `PricingCalculator.java`)

Scenario

A change is proposed on branch `feature/weekend-surcharge` where a weekend surcharge is introduced by modifying the pricing logic in `PricingCalculator.java`.

Instead of running the entire test suite of 49 tests on every change, we select and prioritize which existing tests must be re-run to provide fast, reliable feedback (fail-fast).

1. Change-Relevant Tests (Must Re-run)

1. `PricingCalculatorStructuralTest` **(Priority 1):**
   - **Why:** This directly tests `PricingCalculator`. These unit tests execute in under 20 milliseconds without any database, mocks, or Spring context. If the new weekend logic breaks existing rules (such as free shelter listings, standard walks, or the 480-minute overnight threshold), we find out almost instantly.
2. ▶ `BookingServiceTest` **(Priority 2):**
   - **Why:** `BookingService` is the primary consumer of `PricingCalculator`. It calls `pricing.priceFor(...)` to verify that the seeker has sufficient wallet funds and to charge their balance. Running this ensures that the integration contract between the booking service and pricing remains intact.

2. Wasteful Tests (Do NOT need to run for this change)

Running these tests for a change in `PricingCalculator` wastes developer time and CI computing resources:

- `MatchControllerWebTest` **(Biggest waste):** This test starts the full Spring Boot application and embedded web container, taking 3 to 5 seconds. It tests the web controller for AI match explanations and has no dependency on pricing calculations.
- `MatchExplanationServiceTest` **:** Tests LLM prompt construction and fallback behavior for the AI feature. Completely unrelated to pricing.
- ▶ `SeekerSpecBasedTest` **:** Contains 31 unit tests validating input fields (email format, Swedish phone numbers, wallet bounds) in the `Seeker` domain model. Unaffected by pricing changes.
- `SeekerServiceTest` **:** Tests wallet top-ups and payment gateway errors (declines and timeouts). Does not calculate booking prices.
- `BeginnerFirstTest` **:** Initial sanity test from day one.

3. Recommended Execution Order & Justification

We should execute the selected regression tests in the following order:

1. `PricingCalculatorStructuralTest` (~20 ms)
2. ▶ `BookingServiceTest` (~1 s)

Justification:
Following the *fail-fast* principle, running the fastest and most directly affected unit tests first gives developers immediate feedback within 1 second. Omitting the slow Spring Boot web tests saves significant execution time and CI resources.